

# Sokoban Function Reference

A plain-language guide to the functions used by the project

|              |                                                                                                                 |
|--------------|-----------------------------------------------------------------------------------------------------------------|
| PROJECT      | Sokoban C game                                                                                                  |
| SCOPE        | Every function defined in src/**/*c, plus every external library/standard-library function called by the source |
| ORGANIZATION | Public project API first, internal helpers second, library calls last                                           |
| READING TIP  | The Source column gives a file and line so you can jump from the explanation back to the implementation         |

This document explains what each function contributes to the game. A function is called a project function when it is implemented in src. A library function is an external function supplied by Raylib, the C standard library, the math library, or a compiler built-in.

Scope note: bundled third-party headers are not expanded into this report. Only external functions that the project's own source files actually call are listed.

# 1. Project map

The game is driven by main in src/core/sokoban.c. It initializes assets, audio, settings, and the board; routes input to the active screen; updates gameplay; renders into an off-screen scene; and saves data before shutdown.

| Folder/file         | Role               | Main responsibility                                                                         |
|---------------------|--------------------|---------------------------------------------------------------------------------------------|
| src/core/board.c    | <b>Board rules</b> | ASCII levels, movement, pushes, undo history, solved detection, board rendering             |
| src/core/hint.c     | <b>Hint solver</b> | Bitmask state search, cached push distances, next-push hints                                |
| src/core/score.c    | <b>Scoring</b>     | Stars, push score, move bonus, best records, level unlocks                                  |
| src/core/saves.c    | <b>Persistence</b> | Versioned progress and settings text files                                                  |
| src/audio/audio.c   | <b>Audio</b>       | Sound-effect voices, music streams, mute, volume, ducking, muffle filter                    |
| src/render/assets.c | <b>Assets</b>      | Textures, fonts, and blur shader loading/unloading                                          |
| src/render/ui.c     | <b>Shared UI</b>   | Texture cropping, paper panels, buttons, centered text, hover sound                         |
| src/screens/*.c     | <b>Screens</b>     | Menu, HUD, pause, level select, settings, cheatsheet, controls, credits, scores, completion |

## Typical gameplay flow

Input -> BoardUpdate -> BoardMovePlayer -> CheckSolved -> ScoreFileRun/SaveProgress -> HudDraw/CompletionDraw -> AudioUpdate

## Typical screen flow

Mouse/keyboard input -> screen HitTest/Update helper -> screen Draw helper -> shared UI helpers -> Raylib drawing calls

## 2. User-defined functions - public project API

These 105 functions are declared in project headers and are intended to be called across modules. Each entry gives its exact source location and an easy explanation of its role.

### Audio

Owns sound-effect voices, streamed music, mute/volume settings, and the gameplay music ducking effect.

| Function                         | Signature                                                                | What it does and how the game uses it                                                                         | Source                             |
|----------------------------------|--------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|------------------------------------|
| <b>AudioInit</b>                 | <code>void AudioInit(void)</code>                                        | Opens the audio device, loads sound effects and music, and starts the menu music when available.              | <code>src/audio/audio.c:92</code>  |
| <b>AudioShutdown</b>             | <code>void AudioShutdown(void)</code>                                    | Detaches the music filter and releases every music stream, sound, alias, and the audio device.                | <code>src/audio/audio.c:149</code> |
| <b>AudioUpdate</b>               | <code>void AudioUpdate(float deltaTime, bool paused, bool solved)</code> | Advances music playback, switches volume ducking for pause/solved states, and reduces sound-effect cooldowns. | <code>src/audio/audio.c:177</code> |
| <b>AudioPlay</b>                 | <code>void AudioPlay(SfxId id)</code>                                    | Plays a normal sound effect through the shared voice manager.                                                 | <code>src/audio/audio.c:237</code> |
| <b>AudioPlayPitched</b>          | <code>void AudioPlayPitched(SfxId id, float pitch)</code>                | Plays a sound effect with an extra pitch multiplier, used when a sound needs variation.                       | <code>src/audio/audio.c:238</code> |
| <b>AudioStop</b>                 | <code>void AudioStop(SfxId id)</code>                                    | Stops all active voices for one sound-effect type and clears its cooldown.                                    | <code>src/audio/audio.c:240</code> |
| <b>AudioSetMasterOn</b>          | <code>void AudioSetMasterOn(bool on)</code>                              | Turns all audio on or off by changing Raylib's master volume.                                                 | <code>src/audio/audio.c:249</code> |
| <b>AudioSetMusicVolume</b>       | <code>void AudioSetMusicVolume(float volume)</code>                      | Stores the requested music volume after clamping it to a safe range.                                          | <code>src/audio/audio.c:254</code> |
| <b>AudioSetEffectsVolume</b>     | <code>void AudioSetEffectsVolume(float volume)</code>                    | Stores the requested sound-effects volume after clamping it to a safe range.                                  | <code>src/audio/audio.c:259</code> |
| <b>AudioSetNonGameplayActive</b> | <code>void AudioSetNonGameplayActive(bool active)</code>                 | Switches between menu music and gameplay music when the current screen is not gameplay.                       | <code>src/audio/audio.c:264</code> |
| <b>AudioToggleMute</b>           | <code>void AudioToggleMute(void)</code>                                  | Flips the global mute flag when the player presses the mute shortcut.                                         | <code>src/audio/audio.c:285</code> |
| <b>AudioUnmute</b>               | <code>void AudioUnmute(void)</code>                                      | Clears the global mute flag.                                                                                  | <code>src/audio/audio.c:286</code> |
| <b>AudioIsMuted</b>              | <code>bool AudioIsMuted(void)</code>                                     | Reports whether audio is currently muted.                                                                     | <code>src/audio/audio.c:287</code> |
| <b>AudioMusicIsSilent</b>        | <code>bool AudioMusicIsSilent(void)</code>                               | Reports whether music is effectively silent because it is muted or nearly zero volume.                        | <code>src/audio/audio.c:289</code> |
| <b>AudioRestartMusic</b>         | <code>void AudioRestartMusic(void)</code>                                | Restarts the gameplay music stream, for example after restarting a level.                                     | <code>src/audio/audio.c:294</code> |

### Core / Board

Owns the Sokoban rules: level maps, movement, pushes, undo history, solved detection, and board drawing.

| Function                | Signature                                                               | What it does and how the game uses it                                                                   | Source                            |
|-------------------------|-------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|-----------------------------------|
| <b>BoardSetMovement</b> | <code>void BoardSetMovement(int holdToRepeat, float repeatDelay)</code> | Stores whether movement repeats while a key is held and how long the repeat delay should be.            | <code>src/core/board.c:13</code>  |
| <b>BoardLevelRow</b>    | <code>const char *BoardLevelRow(int levelIndex, int row)</code>         | Returns one ASCII map row for a level, or NULL when the level or row is outside the valid range.        | <code>src/core/board.c:195</code> |
| <b>BoardInit</b>        | <code>void BoardInit(Board *board)</code>                               | Clears a Board, sets its initial facing direction, and loads level zero.                                | <code>src/core/board.c:204</code> |
| <b>BoardLoadLevel</b>   | <code>void BoardLoadLevel(Board *board, int index)</code>               | Resets counters and history, then converts the selected ASCII level into map, box, and player state.    | <code>src/core/board.c:214</code> |
| <b>BoardUpdate</b>      | <code>void BoardUpdate(Board *board, float deltaTime)</code>            | Advances timers, reads directional input, and requests one player move when the repeat timer allows it. | <code>src/core/board.c:261</code> |

| Function         | Signature                                                                                          | What it does and how the game uses it                                                              | Source                            |
|------------------|----------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|-----------------------------------|
| <b>BoardUndo</b> | <code>int BoardUndo(Board *board)</code>                                                           | Reverses the most recent move or push and restores the player's previous facing direction.         | <code>src/core/board.c:418</code> |
| <b>BoardDraw</b> | <code>void BoardDraw(const Board *board, Assets *asset, int originX, int originY, int tile)</code> | Draws visible floor and wall tiles, targets, boxes, and the player at the requested screen origin. | <code>src/core/board.c:443</code> |

## Core / Hint

Builds a cached breadth-first solver and turns the next best push into an on-board hint.

| Function                  | Signature                                                                        | What it does and how the game uses it                                                                | Source                           |
|---------------------------|----------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|----------------------------------|
| <b>HintAsk</b>            | <code>Hint HintAsk(const Board *board)</code>                                    | Analyzes the current board and returns a ready push hint, solved status, or dead/unavailable status. | <code>src/core/hint.c:365</code> |
| <b>HintDraw</b>           | <code>void HintDraw(const Hint *hint, int originX, int originY, int tile)</code> | Draws the animated stand-here, target, arrow, and crate-corner overlay for a ready hint.             | <code>src/core/hint.c:466</code> |
| <b>HintDrawDeadNotice</b> | <code>void HintDrawDeadNotice(Assets *asset, Rectangle area)</code>              | Draws the pulsing 'no way out' notice when the current position is not in the solved-state table.    | <code>src/core/hint.c:514</code> |
| <b>HintUnload</b>         | <code>void HintUnload(void)</code>                                               | Frees all cached hint tables and resets their bookkeeping before shutdown.                           | <code>src/core/hint.c:525</code> |

## Core / Saves

Reads and writes versioned progress and settings files.

| Function            | Signature                                                  | What it does and how the game uses it                                                         | Source                           |
|---------------------|------------------------------------------------------------|-----------------------------------------------------------------------------------------------|----------------------------------|
| <b>SaveProgress</b> | <code>void SaveProgress(int currentLevel)</code>           | Writes the current level and per-level score records to <code>data/progress.txt</code> .      | <code>src/core/saves.c:15</code> |
| <b>LoadProgress</b> | <code>void LoadProgress(int *currentLevel)</code>          | Reads progress data, validates its version, and restores the current level and score records. | <code>src/core/saves.c:30</code> |
| <b>SaveSettings</b> | <code>void SaveSettings(const SettingsState *state)</code> | Writes sound, volume, key-repeat, and speed settings to <code>data/settings.txt</code> .      | <code>src/core/saves.c:73</code> |
| <b>LoadSettings</b> | <code>void LoadSettings(SettingsState *state)</code>       | Reads settings, validates the version, clamps volumes, and restores valid input options.      | <code>src/core/saves.c:87</code> |

## Core / Score

Calculates stars and scores and exposes saved level records to the UI.

| Function                       | Signature                                                    | What it does and how the game uses it                                                                 | Source                           |
|--------------------------------|--------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|----------------------------------|
| <b>GetCompletionStarRating</b> | <code>int GetCompletionStarRating(const Board *board)</code> | Converts the solved level's push count into a 0-to-3 star rating.                                     | <code>src/core/score.c:17</code> |
| <b>GetPushScore</b>            | <code>int GetPushScore(const Board *board)</code>            | Rewards fewer pushes by comparing the run with the level's minimum push target.                       | <code>src/core/score.c:31</code> |
| <b>GetMoveBonus</b>            | <code>int GetMoveBonus(const Board *board)</code>            | Calculates a non-negative bonus that decreases as the total move count rises.                         | <code>src/core/score.c:38</code> |
| <b>GetLevelScore</b>           | <code>int GetLevelScore(const Board *board)</code>           | Adds push score and move bonus for solved boards; unsolved boards score zero.                         | <code>src/core/score.c:44</code> |
| <b>FormatScore</b>             | <code>void FormatScore(char *out, int value)</code>          | Converts an integer score to text and inserts comma separators for display.                           | <code>src/core/score.c:51</code> |
| <b>ScoreFileRun</b>            | <code>bool ScoreFileRun(const Board *board)</code>           | Compares a completed run with the saved record, updates the best score/stars, and reports a new best. | <code>src/core/score.c:67</code> |
| <b>ScoreIsNewBest</b>          | <code>bool ScoreIsNewBest(void)</code>                       | Reports whether the most recently filed run beat the previous score.                                  | <code>src/core/score.c:88</code> |

| Function                  | Signature                                                                   | What it does and how the game uses it                                       | Source               |
|---------------------------|-----------------------------------------------------------------------------|-----------------------------------------------------------------------------|----------------------|
| <b>ScoreSetRecord</b>     | void ScoreSetRecord(int index, int score, int stars, int pushes, int moves) | Loads one validated score record into the in-memory score arrays.           | src/core/score.c:98  |
| <b>ScoreResetAll</b>      | void ScoreResetAll(void)                                                    | Clears every saved score, star, push, and move record.                      | src/core/score.c:109 |
| <b>LevelUnlocked</b>      | bool LevelUnlocked(int index)                                               | Unlocks level zero and any later level whose predecessor has a saved score. | src/core/score.c:121 |
| <b>LevelCleared</b>       | bool LevelCleared(int index)                                                | Reports whether a level has a non-zero saved best score.                    | src/core/score.c:126 |
| <b>LevelMinimumPushes</b> | int LevelMinimumPushes(int index)                                           | Returns the built-in minimum push target for a level.                       | src/core/score.c:131 |
| <b>LevelBestScore</b>     | int LevelBestScore(int index)                                               | Returns a level's saved best score or zero for an invalid index.            | src/core/score.c:136 |
| <b>LevelBestStars</b>     | int LevelBestStars(int index)                                               | Returns a level's saved best star rating or zero for an invalid index.      | src/core/score.c:141 |
| <b>LevelBestPushes</b>    | int LevelBestPushes(int index)                                              | Returns the push count stored with a level's best score.                    | src/core/score.c:146 |
| <b>LevelBestMoves</b>     | int LevelBestMoves(int index)                                               | Returns the move count stored with a level's best score.                    | src/core/score.c:151 |

## Rendering / Assets

Loads and unloads the game's textures, fonts, and blur shader.

| Function            | Signature                        | What it does and how the game uses it                                                       | Source                  |
|---------------------|----------------------------------|---------------------------------------------------------------------------------------------|-------------------------|
| <b>AssetsLoad</b>   | void AssetsLoad(Assets *asset)   | Loads all gameplay, menu, screen, font, and shader assets into the shared Assets structure. | src/render/assets.c:15  |
| <b>AssetsUnload</b> | void AssetsUnload(Assets *asset) | Releases every texture, font, and shader stored in Assets.                                  | src/render/assets.c:106 |

## Rendering / Ui

Provides reusable texture, paper-panel, button, text, cursor, and hover drawing helpers.

| Function                    | Signature                                                                                                                                                | What it does and how the game uses it                                                            | Source              |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|---------------------|
| <b>PauseInk</b>             | Rectangle PauseInk(Texture2D texture, float x, float y, float width, float height)                                                                       | Returns the visible ink rectangle for a pause/menu texture using normalized artwork coordinates. | src/render/ui.c:6   |
| <b>PauseIconInk</b>         | Rectangle PauseIconInk(Texture2D texture, int iconKind)                                                                                                  | Returns the artwork rectangle for a selected pause icon from the icon sheet.                     | src/render/ui.c:14  |
| <b>FitInside</b>            | Rectangle FitInside(Texture2D texture, Rectangle box)                                                                                                    | Calculates the largest rectangle that preserves a texture's aspect ratio inside a box.           | src/render/ui.c:25  |
| <b>DrawWhole</b>            | void DrawWhole(Texture2D texture, Rectangle destination)                                                                                                 | Draws an entire texture into a destination rectangle.                                            | src/render/ui.c:40  |
| <b>DrawInk</b>              | void DrawInk(Texture2D texture, Rectangle ink, Rectangle destination)                                                                                    | Draws only a chosen source rectangle from a texture into a destination rectangle.                | src/render/ui.c:46  |
| <b>DrawPauseButtonBase</b>  | void DrawPauseButtonBase(Texture2D texture, Rectangle destination, Color tint)                                                                           | Draws a pause-style button base with a tint.                                                     | src/render/ui.c:51  |
| <b>DrawPauseIcon</b>        | void DrawPauseIcon(Texture2D texture, Rectangle button, Color tint, int iconKind)                                                                        | Extracts and draws one icon centered inside a button.                                            | src/render/ui.c:92  |
| <b>DrawPauseMenuButton</b>  | void DrawPauseMenuButton(Assets *asset, Rectangle button, Texture2D icon, Texture2D label, Rectangle labelInk, int iconKind, bool hovered, bool enabled) | Composes a pause-menu button from its base, icon, label, hover state, and enabled state.         | src/render/ui.c:112 |
| <b>DrawPaperPlateTinted</b> | void DrawPaperPlateTinted(Texture2D texture, Rectangle ink, Rectangle destination, Color tint)                                                           | Draws a paper-like panel texture with a tint and selected ink rectangle.                         | src/render/ui.c:144 |

| Function                           | Signature                                                                                                                                                    | What it does and how the game uses it                                                      | Source                           |
|------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|----------------------------------|
| <b>DrawPaperPlate</b>              | <code>void DrawPaperPlate(Texture2D texture, Rectangle ink, Rectangle destination)</code>                                                                    | Draws a paper-like panel texture without applying a custom tint.                           | <code>src/render/ui.c:180</code> |
| <b>DrawLitTexture</b>              | <code>void DrawLitTexture(Texture2D texture, Rectangle destination, bool hovered)</code>                                                                     | Draws a texture and adds the shared hover highlight when the control is under the pointer. | <code>src/render/ui.c:185</code> |
| <b>DrawFontCenteredInRectangle</b> | <code>void DrawFontCenteredInRectangle(Font font, const char *text, Rectangle area, float fontSize, float spacing, Color color, float opticalYOffset)</code> | Measures text and draws it centered in a rectangle with an optical vertical adjustment.    | <code>src/render/ui.c:198</code> |
| <b>Draw_Centered_Text</b>          | <code>void Draw_Centered_Text(const char *text, int centerX, int y, int size, Color color)</code>                                                            | Draws a text string centered on a supplied x coordinate.                                   | <code>src/render/ui.c:210</code> |
| <b>UiCursorAndSound</b>            | <code>void UiCursorAndSound(bool overControl)</code>                                                                                                         | Changes the mouse cursor and plays one hover sound when the pointer enters a control.      | <code>src/render/ui.c:217</code> |

## Screens / Cheatsheet

Displays the level route cheatsheets and the reveal/page navigation logic.

| Function                      | Signature                                                                                                                                | What it does and how the game uses it                                                   | Source                                    |
|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|-------------------------------------------|
| <b>CheatsheetInit</b>         | <code>void CheatsheetInit(void)</code>                                                                                                   | Parses and prepares the built-in route data used by cheatsheets and hints.              | <code>src/screens/cheatsheet.c:142</code> |
| <b>CheatsheetRoute</b>        | <code>const CheatRoute *CheatsheetRoute(int levelIndex)</code>                                                                           | Returns the stored route for a level, or NULL when the level is invalid.                | <code>src/screens/cheatsheet.c:152</code> |
| <b>CheatsheetGetLayout</b>    | <code>CheatsheetLayout CheatsheetGetLayout(Assets *asset, float screenWidth, float screenHeight)</code>                                  | Calculates the rectangles used by the cheatsheet screen at the current screen size.     | <code>src/screens/cheatsheet.c:169</code> |
| <b>CheatsheetPageCount</b>    | <code>int CheatsheetPageCount(const CheatsheetLayout *layout, int levelIndex)</code>                                                     | Calculates how many pages are needed to show a level's route in the current layout.     | <code>src/screens/cheatsheet.c:280</code> |
| <b>CheatsheetOpen</b>         | <code>void CheatsheetOpen(CheatsheetState *state, int levelIndex)</code>                                                                 | Starts the cheatsheet at a selected level with its route hidden and page zero selected. | <code>src/screens/cheatsheet.c:288</code> |
| <b>CheatsheetStepLevel</b>    | <code>int CheatsheetStepLevel(CheatsheetState *state, int delta)</code>                                                                  | Moves the cheatsheet selection forward or backward and resets reveal/page state.        | <code>src/screens/cheatsheet.c:299</code> |
| <b>CheatsheetToggleReveal</b> | <code>void CheatsheetToggleReveal(CheatsheetState *state)</code>                                                                         | Shows or hides the selected route.                                                      | <code>src/screens/cheatsheet.c:308</code> |
| <b>CheatsheetStepPage</b>     | <code>int CheatsheetStepPage(CheatsheetState *state, const CheatsheetLayout *layout, int delta)</code>                                   | Moves the cheatsheet page within its valid range.                                       | <code>src/screens/cheatsheet.c:316</code> |
| <b>CheatsheetHitTest</b>      | <code>CheatsheetControl CheatsheetHitTest(const CheatsheetLayout *layout, const CheatsheetState *state, Vector2 mouse)</code>            | Identifies which cheatsheet tab or button is under the mouse.                           | <code>src/screens/cheatsheet.c:334</code> |
| <b>CheatsheetDraw</b>         | <code>void CheatsheetDraw(Assets *asset, const CheatsheetLayout *layout, const CheatsheetState *state, CheatsheetControl hovered)</code> | Renders the complete cheatsheet page, controls, route, and locked/revealed state.       | <code>src/screens/cheatsheet.c:601</code> |

## Screens / Completion

Displays score results and animated stars after a level is solved.

| Function                   | Signature                                                                                                                                                          | What it does and how the game uses it                                                | Source                                    |
|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|-------------------------------------------|
| <b>CompletionGetLayout</b> | <code>CompletionLayout CompletionGetLayout(void)</code>                                                                                                            | Returns the fixed positions for the completion panel and its three buttons.          | <code>src/screens/completion.c:8</code>   |
| <b>CompletionDraw</b>      | <code>void CompletionDraw(const Board *board, Assets *asset, CompletionLayout layout, bool nextHovered, bool replayHovered, bool homeHovered, float reveal)</code> | Renders the completion artwork, score details, stars, and next/replay/home controls. | <code>src/screens/completion.c:123</code> |

| Function                        | Signature                                                                                      | What it does and how the game uses it                                         | Source                                    |
|---------------------------------|------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|-------------------------------------------|
| <b>CompletionPlayStartTicks</b> | <code>void CompletionPlayStartTicks(const Board *board, float reveal, int *starsTicked)</code> | Plays star sounds as the animated completion reveal reaches each earned star. | <code>src/screens/completion.c:300</code> |

## Screens / Controls

Displays the keyboard and mouse controls reference screen.

| Function                 | Signature                                                                                            | What it does and how the game uses it                               | Source                                  |
|--------------------------|------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|-----------------------------------------|
| <b>ControlsGetLayout</b> | <code>ControlsLayout ControlsGetLayout(Assets *asset, float screenWidth, float screenHeight)</code>  | Calculates the controls screen artwork rectangles.                  | <code>src/screens/controls.c:58</code>  |
| <b>ControlsHitTest</b>   | <code>ControlsControl ControlsHitTest(const ControlsLayout *layout, Vector2 mouse)</code>            | Reports whether the controls screen back button is under the mouse. | <code>src/screens/controls.c:89</code>  |
| <b>ControlsDraw</b>      | <code>void ControlsDraw(Assets *asset, const ControlsLayout *layout, ControlsControl hovered)</code> | Renders the controls reference screen and back control.             | <code>src/screens/controls.c:258</code> |

## Screens / Credits

Displays the credits sheet.

| Function                | Signature                                                                                         | What it does and how the game uses it                                       | Source                                 |
|-------------------------|---------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|----------------------------------------|
| <b>CreditsGetLayout</b> | <code>CreditsLayout CreditsGetLayout(Assets *asset, float screenWidth, float screenHeight)</code> | Calculates the credits screen artwork rectangles.                           | <code>src/screens/credits.c:66</code>  |
| <b>CreditsHitTest</b>   | <code>CreditsControl CreditsHitTest(const CreditsLayout *layout, Vector2 mouse)</code>            | Reports whether the credits screen back button is under the mouse.          | <code>src/screens/credits.c:97</code>  |
| <b>CreditsDraw</b>      | <code>void CreditsDraw(Assets *asset, const CreditsLayout *layout, CreditsControl hovered)</code> | Renders the credits sheet, headings, names, music credit, and back control. | <code>src/screens/credits.c:199</code> |

## Screens / Highscores

Displays paged saved score records.

| Function                   | Signature                                                                                                                                | What it does and how the game uses it                                      | Source                                    |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|-------------------------------------------|
| <b>HighScoresPageCount</b> | <code>int HighScoresPageCount(void)</code>                                                                                               | Returns the number of high-score pages based on the number of levels.      | <code>src/screens/highscores.c:23</code>  |
| <b>HighScoresOpen</b>      | <code>void HighScoresOpen(HighScoresState *state, int levelIndex)</code>                                                                 | Opens high scores at the page containing the selected level.               | <code>src/screens/highscores.c:28</code>  |
| <b>HighScoresStepPage</b>  | <code>int HighScoresStepPage(HighScoresState *state, int delta)</code>                                                                   | Moves the high-score page while clamping it to valid pages.                | <code>src/screens/highscores.c:37</code>  |
| <b>HighScoresGetLayout</b> | <code>HighScoresLayout HighScoresGetLayout(Assets *asset, float screenWidth, float screenHeight)</code>                                  | Calculates the score table, tabs, columns, and decorative rules.           | <code>src/screens/highscores.c:46</code>  |
| <b>HighScoresHitTest</b>   | <code>HighScoresControl HighScoresHitTest(const HighScoresLayout *layout, const HighScoresState *state, Vector2 mouse)</code>            | Identifies the high-score previous, next, or back control under the mouse. | <code>src/screens/highscores.c:120</code> |
| <b>HighScoresDraw</b>      | <code>void HighScoresDraw(Assets *asset, const HighScoresLayout *layout, const HighScoresState *state, HighScoresControl hovered)</code> | Renders the high-score table, page marker, and navigation controls.        | <code>src/screens/highscores.c:285</code> |

## Screens / Hud

Displays the gameplay board frame, statistics, hint state, and action buttons.

| Function            | Signature                                                                                                               | What it does and how the game uses it                                              | Source                             |
|---------------------|-------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|------------------------------------|
| <b>HudGetLayout</b> | <code>HudLayout HudGetLayout(void)</code>                                                                               | Builds the gameplay HUD positions, board origin, tile size, and button rectangles. | <code>src/screens/hud.c:9</code>   |
| <b>HudDraw</b>      | <code>void HudDraw(const Board *board, Assets *asset, const HudLayout *layout, const Hint *hint, HudHover hover)</code> | Renders the gameplay background, board, stats, hint message, and HUD controls.     | <code>src/screens/hud.c:232</code> |

## Screens / Levelselect

Displays level cards, locked progress, and the high-score/cheatsheet shelves.

| Function                          | Signature                                                                                                                     | What it does and how the game uses it                                                       | Source                                     |
|-----------------------------------|-------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|--------------------------------------------|
| <b>LevelSelectSlotCount</b>       | <code>int LevelSelectSlotCount(void)</code>                                                                                   | Returns the number of level cards shown by level select.                                    | <code>src/screens/levelselect.c:8</code>   |
| <b>LevelSelectGetLayout</b>       | <code>LevelSelectLayout LevelSelectGetLayout(Assets *asset)</code>                                                            | Calculates level cards and bottom-shelf button rectangles.                                  | <code>src/screens/levelselect.c:13</code>  |
| <b>LevelSelectHitTest</b>         | <code>LevelSelectHover LevelSelectHitTest(const LevelSelectLayout *layout, Vector2 mouse)</code>                              | Identifies a level card, locked card, shelf button, or back/control button under the mouse. | <code>src/screens/levelselect.c:71</code>  |
| <b>LevelSelectHoveringControl</b> | <code>bool LevelSelectHoveringControl(LevelSelectHover hover)</code>                                                          | Returns true when a level-select control, shelf, or locked card is being hovered.           | <code>src/screens/levelselect.c:97</code>  |
| <b>LevelSelectDraw</b>            | <code>void LevelSelectDraw(const Board *board, Assets *asset, const LevelSelectLayout *layout, LevelSelectHover hover)</code> | Renders the level-select background, banner, cards, and shelf controls.                     | <code>src/screens/levelselect.c:236</code> |

## Screens / Menu

Displays the main menu and maps pointer positions to menu items.

| Function             | Signature                                                                                            | What it does and how the game uses it                                             | Source                             |
|----------------------|------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|------------------------------------|
| <b>MenuGetLayout</b> | <code>MenuLayout MenuGetLayout(Assets *asset)</code>                                                 | Calculates main-menu title, panel, buttons, and credits hit areas.                | <code>src/screens/menu.c:44</code> |
| <b>MenuHitTest</b>   | <code>int MenuHitTest(const MenuLayout *layout, Vector2 mouse)</code>                                | Returns the main-menu item under the mouse, or a negative value when none is hit. | <code>src/screens/menu.c:86</code> |
| <b>MenuDraw</b>      | <code>void MenuDraw(const Board *board, Assets *asset, const MenuLayout *layout, int hovered)</code> | Renders the menu background, title, buttons, and continue state.                  | <code>src/screens/menu.c:98</code> |

## Screens / Pause

Displays the pause overlay and its action buttons.

| Function              | Signature                                                                                                                                                                  | What it does and how the game uses it                                      | Source                              |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|-------------------------------------|
| <b>PauseGetLayout</b> | <code>PauseLayout PauseGetLayout(Assets *asset)</code>                                                                                                                     | Calculates the pause panel and its four action buttons.                    | <code>src/screens/pause.c:7</code>  |
| <b>PauseDraw</b>      | <code>void PauseDraw(const Board *board, Assets *asset, const PauseLayout *layout, bool resumeHovered, bool restartHovered, bool settingsHovered, bool homeHovered)</code> | Renders the pause panel, status, and resume/restart/settings/home buttons. | <code>src/screens/pause.c:87</code> |

## Screens / Settings

Displays and updates audio and input settings, including reset confirmation.

| Function                 | Signature                                                    | What it does and how the game uses it                                    | Source                                 |
|--------------------------|--------------------------------------------------------------|--------------------------------------------------------------------------|----------------------------------------|
| <b>SettingsGetLayout</b> | <code>SettingsLayout SettingsGetLayout(Assets *asset)</code> | Calculates tabs, sliders, reset confirmation, and navigation rectangles. | <code>src/screens/settings.c:92</code> |

| Function                   | Signature                                                                                                     | What it does and how the game uses it                                                                        | Source                                  |
|----------------------------|---------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|-----------------------------------------|
| <b>SettingsRepeatDelay</b> | <code>float SettingsRepeatDelay(int speedIndex)</code>                                                        | Maps the selected speed index to a keyboard repeat delay.                                                    | <code>src/screens/settings.c:344</code> |
| <b>SettingsDefaults</b>    | <code>void SettingsDefaults(SettingsState *state)</code>                                                      | Fills a SettingsState with the game's default audio and input values.                                        | <code>src/screens/settings.c:351</code> |
| <b>SettingsApply</b>       | <code>void SettingsApply(const SettingsState *state)</code>                                                   | Applies settings to audio and board movement behavior.                                                       | <code>src/screens/settings.c:367</code> |
| <b>SettingsDraw</b>        | <code>void SettingsDraw(Assets *asset, const SettingsLayout *layout, const SettingsState *state)</code>       | Renders the settings screen and its optional reset confirmation dialog.                                      | <code>src/screens/settings.c:376</code> |
| <b>SettingsUpdate</b>      | <code>SettingsResult SettingsUpdate(const SettingsLayout *layout, SettingsState *state, Vector2 mouse)</code> | Processes settings mouse interactions, dragging, reset confirmation, and closing; returns the screen result. | <code>src/screens/settings.c:385</code> |

## 3. User-defined functions - internal helpers

These 85 functions are implemented inside source files and are not declared as cross-module API. They support the public functions above: validation, layout math, drawing composition, parsing, search, and small reusable operations.

### Audio

Owns sound-effect voices, streamed music, mute/volume settings, and the gameplay music ducking effect.

| Function               | Signature                                                                   | What it does and how the game uses it                                                                    | Source                             |
|------------------------|-----------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|------------------------------------|
| <b>ClampVolume</b>     | <code>static float ClampVolume(float v)</code>                              | Keeps a volume value between 0.0 and 1.0 before it is stored.                                            | <code>src/audio/audio.c:6</code>   |
| <b>MusicRoomFilter</b> | <code>static void MusicRoomFilter(void *buffer, unsigned int frames)</code> | Audio callback that applies a simple smoothing filter so gameplay music sounds muffled when ducked.      | <code>src/audio/audio.c:71</code>  |
| <b>PlayVoice</b>       | <code>static void PlayVoice(SfxId id, float pitch)</code>                   | Chooses the next voice for a sound effect, applies volume, pitch variation, and cooldown, then plays it. | <code>src/audio/audio.c:215</code> |

### Core / Board

Owns the Sokoban rules: level maps, movement, pushes, undo history, solved detection, and board drawing.

| Function               | Signature                                                                                              | What it does and how the game uses it                                                                      | Source                            |
|------------------------|--------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|-----------------------------------|
| <b>DirectionHeld</b>   | <code>static int DirectionHeld(int keyA, int keyB)</code>                                              | Chooses held-key or pressed-key input according to the movement setting.                                   | <code>src/core/board.c:19</code>  |
| <b>InsideGrid</b>      | <code>int InsideGrid(int r, int c)</code>                                                              | Checks whether a row and column are inside the fixed board dimensions.                                     | <code>src/core/board.c:303</code> |
| <b>CheckSolved</b>     | <code>int CheckSolved(Board*board)</code>                                                              | Returns true only when every box is sitting on a target tile.                                              | <code>src/core/board.c:312</code> |
| <b>BoardMovePlayer</b> | <code>static void BoardMovePlayer(Board *board,int rowChange, int colChange)</code>                    | Validates a step or push, updates positions and counters, records undo data, and plays the matching sound. | <code>src/core/board.c:325</code> |
| <b>DrawTile</b>        | <code>static void DrawTile(Texture2D tex, float dest_x, float dest_y, float tile)</code>               | Draws one texture scaled to a square board tile.                                                           | <code>src/core/board.c:492</code> |
| <b>DrawWallTile</b>    | <code>static void DrawWallTile(Assets *asset,int dest_x, int dest_y, int tile)</code>                  | Draws the wall-cap texture for one wall cell.                                                              | <code>src/core/board.c:501</code> |
| <b>DrawGoalTile</b>    | <code>static void DrawGoalTile(Assets * asset,int dest_x, int dest_y, int tile)</code>                 | Draws the target marker on a walkable board cell.                                                          | <code>src/core/board.c:506</code> |
| <b>DrawBox</b>         | <code>static void DrawBox(Assets * asset,int dest_x, int dest_y, int tile, int onTarget)</code>        | Selects the normal or completed-box texture and draws it at the requested cell.                            | <code>src/core/board.c:511</code> |
| <b>DrawPlayer</b>      | <code>static void DrawPlayer(const Board*board,Assets * asset,int dest_x, int dest_y, int tile)</code> | Selects the player's facing and push-pose texture, centers it in a tile, and draws it.                     | <code>src/core/board.c:519</code> |

### Core / Hint

Builds a cached breadth-first solver and turns the next best push into an on-board hint.

| Function         | Signature                                                                                                           | What it does and how the game uses it                                                   | Source                          |
|------------------|---------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|---------------------------------|
| <b>HintHash</b>  | <code>static size_t HintHash(uint64_t mask, uint16_t player)</code>                                                 | Hashes a crate bitmask plus player position for the hint solver's open-addressed table. | <code>src/core/hint.c:48</code> |
| <b>HintFind</b>  | <code>static HintEntry *HintFind(HintLevel *lv, uint64_t mask, uint16_t player)</code>                              | Looks up one solver state in the cached hash table.                                     | <code>src/core/hint.c:58</code> |
| <b>HintPlace</b> | <code>static void HintPlace(HintEntry *slot, size_t capacity, uint64_t mask, uint16_t player, uint16_t dist)</code> | Places a state directly into a hash table while rebuilding that table.                  | <code>src/core/hint.c:74</code> |
| <b>HintGrow</b>  | <code>static bool HintGrow(HintLevel *lv)</code>                                                                    | Doubles the solver table capacity and rehashes its existing entries.                    | <code>src/core/hint.c:86</code> |

| Function              | Signature                                                                            | What it does and how the game uses it                                                                    | Source              |
|-----------------------|--------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|---------------------|
| <b>HintInsert</b>     | static bool HintInsert(HintLevel *lv, uint64_t mask, uint16_t player, uint16_t dist) | Adds a new solver state unless the same state already exists; grows the table when it is getting full.   | src/core/hint.c:106 |
| <b>HintReach</b>      | static uint64_t HintReach(const HintLevel *lv, uint64_t crates, int from)            | Flood-fills all walkable cells the player can reach without moving crates.                               | src/core/hint.c:130 |
| <b>HintNormalise</b>  | static int HintNormalise(const HintLevel *lv, uint64_t crates, int from)             | Converts a reachable region into one canonical player cell so equivalent positions share a solver state. | src/core/hint.c:155 |
| <b>HintGeometry</b>   | static bool HintGeometry(HintLevel *lv, int level)                                   | Builds the walkable-cell index, neighbors, goal mask, and starting crate/player data for one level.      | src/core/hint.c:160 |
| <b>HintBuildRoute</b> | static void HintBuildRoute(HintLevel *lv, int level)                                 | Simulates the saved cheatsheet route and records states that belong to that preferred route.             | src/core/hint.c:219 |
| <b>HintOnRoute</b>    | static bool HintOnRoute(const HintLevel *lv, uint64_t mask, uint16_t player)         | Checks whether a solver state matches one of the saved route states.                                     | src/core/hint.c:265 |
| <b>HintQueuePush</b>  | static void HintQueuePush(HintQueue *q, uint64_t mask, int player)                   | Appends a state to the dynamically growing breadth-first-search queue.                                   | src/core/hint.c:282 |
| <b>HintBuildTable</b> | static bool HintBuildTable(HintLevel *lv, int level)                                 | Builds the reverse push-distance table used to find a shortest remaining push sequence.                  | src/core/hint.c:305 |

## Core / Score

Calculates stars and scores and exposes saved level records to the UI.

| Function            | Signature                           | What it does and how the game uses it                            | Source              |
|---------------------|-------------------------------------|------------------------------------------------------------------|---------------------|
| <b>IndexInRange</b> | static bool IndexInRange(int index) | Checks whether a level index can safely access the score arrays. | src/core/score.c:93 |

## Core / Sokoban

Contains the application entry point and the helper that changes levels.

| Function         | Signature                                      | What it does and how the game uses it                                                                      | Source                |
|------------------|------------------------------------------------|------------------------------------------------------------------------------------------------------------|-----------------------|
| <b>GoToLevel</b> | static void GoToLevel(Board *board, int index) | Stops completion audio, loads a wrapped level index, and restarts gameplay music.                          | src/core/sokoban.c:25 |
| <b>main</b>      | int main(void)                                 | Initializes the window and systems, runs the screen/gameplay loop, saves state, and shuts everything down. | src/core/sokoban.c:32 |

## Rendering / Assets

Loads and unloads the game's textures, fonts, and blur shader.

| Function        | Signature                                | What it does and how the game uses it                                                          | Source                |
|-----------------|------------------------------------------|------------------------------------------------------------------------------------------------|-----------------------|
| <b>LoadTile</b> | Texture2D LoadTile(const char *fileName) | Loads one image, generates mipmaps, and applies trilinear filtering when the texture is valid. | src/render/assets.c:4 |

## Screens / Cheatsheet

Displays the level route cheatsheets and the reveal/page navigation logic.

| Function          | Signature                                                   | What it does and how the game uses it                                             | Source                       |
|-------------------|-------------------------------------------------------------|-----------------------------------------------------------------------------------|------------------------------|
| <b>ParseRoute</b> | static void ParseRoute(const char *text, CheatRoute *route) | Parses a compact route string into repeated directional cheat steps.              | src/screens/cheatsheet.c:110 |
| <b>GridOf</b>     | static RouteGrid GridOf(const CheatsheetLayout *layout)     | Converts the cheatsheet paper rectangle into a grid of cells for route rendering. | src/screens/cheatsheet.c:253 |

| Function                     | Signature                                                                                                                      | What it does and how the game uses it                                | Source                       |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------|------------------------------|
| <b>DrawCheatTextCentered</b> | static void DrawCheatTextCentered(Font font, const char *text, float centerX, float centerY, float size, Color color)          | Draws cheatsheet text centered on a position.                        | src/screens/cheatsheet.c:366 |
| <b>DrawWholeTinted</b>       | static void DrawWholeTinted(Texture2D texture, Rectangle destination, Color tint)                                              | Draws an entire texture with a supplied tint.                        | src/screens/cheatsheet.c:377 |
| <b>DrawDirectionArrow</b>    | static void DrawDirectionArrow(Texture2D arrow, Rectangle box, int direction, Color tint)                                      | Draws a rotated directional arrow for one route step.                | src/screens/cheatsheet.c:384 |
| <b>DrawCheatButton</b>       | static void DrawCheatButton(Assets *asset, Rectangle button, const char *label, bool hovered, bool enabled)                    | Draws a cheatsheet control with its label and hover state.           | src/screens/cheatsheet.c:405 |
| <b>DrawNavigationTab</b>     | static void DrawNavigationTab(Assets *asset, Rectangle tab, const char *label, int direction, bool hovered, bool enabled)      | Draws a previous/next navigation tab.                                | src/screens/cheatsheet.c:425 |
| <b>DrawRouteRow</b>          | static void DrawRouteRow(Assets *asset, const CheatRoute *route, const RouteGrid *grid, int row, int firstStep, int rowNumber) | Draws one row of route steps, including arrows and repeat counts.    | src/screens/cheatsheet.c:458 |
| <b>DrawRoute</b>             | static void DrawRoute(Assets *asset, const CheatsheetLayout *layout, const CheatsheetState *state, CheatsheetControl hovered)  | Draws the visible portion of the selected route page.                | src/screens/cheatsheet.c:507 |
| <b>DrawLockedPaper</b>       | static void DrawLockedPaper(Assets *asset, const CheatsheetLayout *layout)                                                     | Draws the locked state shown before a cheatsheet is revealed.        | src/screens/cheatsheet.c:548 |
| <b>DrawRedactedPaper</b>     | static void DrawRedactedPaper(Assets *asset, const CheatsheetLayout *layout)                                                   | Draws the redacted/covered paper treatment for hidden route content. | src/screens/cheatsheet.c:578 |

## Screens / Completion

Displays score results and animated stars after a level is solved.

| Function                   | Signature                                                                                                                        | What it does and how the game uses it                             | Source                       |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------|------------------------------|
| <b>DrawStar</b>            | static void DrawStar(Texture2D star, Rectangle ink, Rectangle box, Color tint)                                                   | Draws one filled or empty star with the requested tint.           | src/screens/completion.c:39  |
| <b>DrawCompletionStars</b> | static void DrawCompletionStars(Assets *asset, Rectangle panel, int starRating, float reveal)                                    | Draws the star row for the completed level with reveal animation. | src/screens/completion.c:61  |
| <b>DrawCompletionRow</b>   | static void DrawCompletionRow(Font font, const char *label, const char *value, float left, float right, float y, float fontSize) | Draws one label/value row in the completion panel.                | src/screens/completion.c:110 |

## Screens / Controls

Displays the keyboard and mouse controls reference screen.

| Function                   | Signature                                                                                                           | What it does and how the game uses it                              | Source                     |
|----------------------------|---------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|----------------------------|
| <b>DrawRowText</b>         | static void DrawRowText(Font font, const char *text, float x, float centerY, float size, Color color)               | Draws one left-aligned credits row.                                | src/screens/controls.c:103 |
| <b>DrawRowTextCentered</b> | static void DrawRowTextCentered(Font font, const char *text, float centerX, float centerY, float size, Color color) | Draws one credits row centered on an x coordinate.                 | src/screens/controls.c:112 |
| <b>DrawArrowGlyph</b>      | static void DrawArrowGlyph(Texture2D arrow, Rectangle box, float rotation, Color tint)                              | Draws a directional arrow glyph from the cheatsheet arrow texture. | src/screens/controls.c:123 |
| <b>CapWidth</b>            | static float CapWidth(Font font, const char *label, float height)                                                   | Calculates a key-cap width from its label and desired height.      | src/screens/controls.c:136 |

| Function            | Signature                                                                                                           | What it does and how the game uses it                                | Source                     |
|---------------------|---------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------|----------------------------|
| <b>DrawKeyCap</b>   | static void DrawKeyCap(Assets *asset, Rectangle cap, const char *label)                                             | Draws a key-cap panel and its label.                                 | src/screens/controls.c:151 |
| <b>RunKeyColumn</b> | static float RunKeyColumn(Assets *asset, const ControlRow *row, float x, float centerY, float capHeight, bool draw) | Lays out one column of control keys and returns the next x position. | src/screens/controls.c:189 |
| <b>DrawBlock</b>    | static void DrawBlock(Assets *asset, const ControlBlock *block, Rectangle area, float unit)                         | Draws one structured credits block and its rows.                     | src/screens/controls.c:221 |

## Screens / Credits

Displays the credits sheet.

| Function                | Signature                                                                                                       | What it does and how the game uses it                          | Source                    |
|-------------------------|-----------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------|---------------------------|
| <b>SheetUnits</b>       | static float SheetUnits(void)                                                                                   | Returns the base unit used by the credits sheet layout.        | src/screens/credits.c:53  |
| <b>DrawRowTextRight</b> | static void DrawRowTextRight(Font font, const char *text, float rightX, float centerY, float size, Color color) | Draws one credits row aligned to a right edge.                 | src/screens/credits.c:120 |
| <b>FitTextSize</b>      | static float FitTextSize(Font font, const char *text, float wanted, float maxWidth)                             | Reduces text size until a label fits within a requested width. | src/screens/credits.c:141 |

## Screens / Highscores

Displays paged saved score records.

| Function                     | Signature                                                                                                               | What it does and how the game uses it                     | Source                       |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|------------------------------|
| <b>DrawSheetTextCentered</b> | static void DrawSheetTextCentered(Font font, const char *text, float centerX, float centerY, float size, Color color)   | Draws table text centered within a score-sheet column.    | src/screens/highscores.c:139 |
| <b>DrawNoRecordDash</b>      | static void DrawNoRecordDash(float centerX, float centerY, float size, Color color)                                     | Draws a dash in a table cell that has no saved record.    | src/screens/highscores.c:150 |
| <b>DrawWholeTexture</b>      | static void DrawWholeTexture(Texture2D texture, Rectangle destination, Color tint)                                      | Draws a whole texture with a caller-selected tint.        | src/screens/highscores.c:159 |
| <b>DrawPageTab</b>           | static void DrawPageTab(Assets *asset, Rectangle tab, const char *label, bool pointingLeft, bool hovered, bool enabled) | Draws a high-score page tab and its hover state.          | src/screens/highscores.c:166 |
| <b>DrawRatingStars</b>       | static void DrawRatingStars(Assets *asset, Rectangle cell, int stars, float opacity)                                    | Draws the saved star rating inside one high-score cell.   | src/screens/highscores.c:208 |
| <b>DrawCaseRow</b>           | static void DrawCaseRow(Assets *asset, const HighScoresLayout *layout, int row, int level)                              | Draws one level row with score, stars, pushes, and moves. | src/screens/highscores.c:227 |

## Screens / Hud

Displays the gameplay board frame, statistics, hint state, and action buttons.

| Function                  | Signature                                                                             | What it does and how the game uses it                   | Source                |
|---------------------------|---------------------------------------------------------------------------------------|---------------------------------------------------------|-----------------------|
| <b>DrawGameBackground</b> | static void DrawGameBackground(Assets *asset)                                         | Draws the gameplay background texture behind the board. | src/screens/hud.c:83  |
| <b>DrawHudPanel</b>       | static void DrawHudPanel(Assets *asset, Rectangle destination)                        | Draws a HUD panel texture into a rectangle.             | src/screens/hud.c:101 |
| <b>DrawTextInPanel</b>    | static void DrawTextInPanel(const char *text, Rectangle panel, int size, Color color) | Centers a stat label/value inside a HUD panel.          | src/screens/hud.c:145 |

| Function                | Signature                                                                                                               | What it does and how the game uses it                      | Source                |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------|-----------------------|
| <b>DrawUiIcon</b>       | static void DrawUiIcon(Assets *asset, int column, int row, Rectangle destination, Color tint)                           | Extracts and draws one icon from the shared UI icon sheet. | src/screens/hud.c:153 |
| <b>DrawUIButtonLift</b> | static void DrawUIButtonLift(Assets *asset, int iconColumn, int iconRow, Rectangle destination, Color lift)             | Draws a HUD button with a lifted hover treatment.          | src/screens/hud.c:187 |
| <b>DrawUIButton</b>     | static void DrawUIButton(Assets *asset, int iconColumn, int iconRow, Rectangle destination, bool enabled, bool hovered) | Draws a HUD button and its icon without the hover lift.    | src/screens/hud.c:208 |

## Screens / Levelselect

Displays level cards, locked progress, and the high-score/cheatsheet shelves.

| Function               | Signature                                                                                                     | What it does and how the game uses it                                 | Source                        |
|------------------------|---------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|-------------------------------|
| <b>DrawBoardButton</b> | static void DrawBoardButton(Assets *asset, Rectangle button, Texture2D icon, const char *label, bool hovered) | Draws a bottom-shelf button with an icon and hover treatment.         | src/screens/levelselect.c:102 |
| <b>DrawLevelStars</b>  | static void DrawLevelStars(Assets *asset, Rectangle card, int stars)                                          | Draws the saved star rating beside a level card.                      | src/screens/levelselect.c:140 |
| <b>LiftLevelCard</b>   | static void LiftLevelCard(Rectangle card)                                                                     | Applies the small hover offset used to lift a level card.             | src/screens/levelselect.c:160 |
| <b>DrawLevelCard</b>   | static void DrawLevelCard(const Board *board, Assets *asset, Rectangle card, int index, bool hovered)         | Draws one unlocked or locked level card and its progress information. | src/screens/levelselect.c:167 |

## Screens / Menu

Displays the main menu and maps pointer positions to menu items.

| Function              | Signature                                                                                                                              | What it does and how the game uses it                | Source               |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------|----------------------|
| <b>DrawMenuButton</b> | static void DrawMenuButton(Assets *asset, Rectangle button, Texture2D label, Texture2D icon, int iconKind, bool hasIcon, bool hovered) | Draws a main-menu button label with its hover state. | src/screens/menu.c:7 |

## Screens / Pause

Displays the pause overlay and its action buttons.

| Function               | Signature                                                                                                              | What it does and how the game uses it               | Source                 |
|------------------------|------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|------------------------|
| <b>DrawPauseStatus</b> | static void DrawPauseStatus(Font font, const Board *board, Rectangle area, float fontSize, float spacing, Color color) | Draws the paused level number and progress summary. | src/screens/pause.c:45 |

## Screens / Settings

Displays and updates audio and input settings, including reset confirmation.

| Function                 | Signature                                                                                                       | What it does and how the game uses it                                           | Source                    |
|--------------------------|-----------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|---------------------------|
| <b>SettingsRect</b>      | static Rectangle SettingsRect(float x, float y, float w, float h)                                               | Creates a Rectangle value from four float coordinates for settings layout code. | src/screens/settings.c:21 |
| <b>LiftPaperPlate</b>    | static void LiftPaperPlate(Texture2D texture, Rectangle ink, Rectangle destination)                             | Draws a settings plate with a hover lift offset.                                | src/screens/settings.c:35 |
| <b>SettingsCapSize</b>   | static float SettingsCapSize(float capHeight)                                                                   | Calculates the width of a settings key-cap based on its height.                 | src/screens/settings.c:47 |
| <b>DrawSettingsLabel</b> | static void DrawSettingsLabel(Font font, const char *text, float x, float capTop, float capHeight, Color color) | Draws a left-aligned settings label.                                            | src/screens/settings.c:52 |

| Function                         | Signature                                                                                                                                                                                                                | What it does and how the game uses it                                | Source                     |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------|----------------------------|
| <b>DrawSettingsLabelCentered</b> | static void DrawSettingsLabelCentered(Font font, const char *text, float centerX, float capTop, float capHeight, Color color)                                                                                            | Draws a centered settings label.                                     | src/screens/settings.c:65  |
| <b>DrawSettingsLabelInRect</b>   | static void DrawSettingsLabelInRect(Font font, const char *text, Rectangle rect, float capHeight, Color color)                                                                                                           | Draws a settings label centered inside a rectangle.                  | src/screens/settings.c:79  |
| <b>SliderInset</b>               | static float SliderInset(Rectangle track)                                                                                                                                                                                | Returns the usable inner track rectangle for a settings slider.      | src/screens/settings.c:113 |
| <b>SliderKnobX</b>               | static float SliderKnobX(Rectangle track, float value)                                                                                                                                                                   | Converts a 0-to-1 slider value into a knob x position.               | src/screens/settings.c:118 |
| <b>SliderValueAt</b>             | static float SliderValueAt(Rectangle track, float mouseX)                                                                                                                                                                | Converts a mouse x position on a slider into a clamped 0-to-1 value. | src/screens/settings.c:124 |
| <b>DrawSettingsSlider</b>        | static void DrawSettingsSlider(Assets *asset, Rectangle track, float value, bool hovered)                                                                                                                                | Draws a slider track, fill, and knob.                                | src/screens/settings.c:131 |
| <b>DrawSettingsOption</b>        | static void DrawSettingsOption(Assets *asset, Rectangle segment, const char *text, bool selected, bool hovered)                                                                                                          | Draws one selected/unselected settings option segment.               | src/screens/settings.c:167 |
| <b>ConfirmPanelRect</b>          | static Rectangle ConfirmPanelRect(Assets *asset)                                                                                                                                                                         | Returns the rectangle for the reset-confirmation panel.              | src/screens/settings.c:188 |
| <b>ConfirmButton</b>             | static Rectangle ConfirmButton(Rectangle panel, int index)                                                                                                                                                               | Returns the rectangle for one button inside the confirmation panel.  | src/screens/settings.c:197 |
| <b>DrawConfirmPlate</b>          | static void DrawConfirmPlate(Assets *asset, Rectangle button, const char *label, bool erase, bool hovered, bool pressed)                                                                                                 | Draws one reset-confirmation button with hover and pressed styling.  | src/screens/settings.c:206 |
| <b>DrawSettingsSheet</b>         | static void DrawSettingsSheet(Assets *asset, SettingsLayout layout, bool masterSoundOn, float musicVolume, float effectsVolume, int holdToRepeat, int speedIndex, int resetState, bool showConfirm, SettingsHover hover) | Draws the main settings sheet, options, sliders, and reset controls. | src/screens/settings.c:229 |

## 4. Library functions used by the project

These functions are not implemented by the project. They come from Raylib, the C standard library, the math library, or compiler built-ins. The descriptions focus on the exact way this code uses them.

Library calls documented: 89

### Raylib - window, input, timing

| Library function                    | Simple use in this project                                                         |
|-------------------------------------|------------------------------------------------------------------------------------|
| <code>InitWindow</code>             | Creates the game window.                                                           |
| <code>CloseWindow</code>            | Closes the game window at shutdown.                                                |
| <code>WindowShouldClose</code>      | Tells the main loop when the player or window requests exit.                       |
| <code>SetTargetFPS</code>           | Requests the game's 60 FPS target.                                                 |
| <code>GetFrameTime</code>           | Returns elapsed seconds for frame-based timers and animation.                      |
| <code>GetTime</code>                | Returns elapsed application time for pulsing hint graphics.                        |
| <code>GetMousePosition</code>       | Reads the current pointer position for hit testing.                                |
| <code>GetMouseWheelMove</code>      | Reads the scroll-wheel delta for paged screens.                                    |
| <code>IsKeyDown</code>              | Reports a key held this frame; used for repeatable movement.                       |
| <code>IsKeyPressed</code>           | Reports a key pressed this frame; used for one-shot actions and optional movement. |
| <code>IsMouseButtonDown</code>      | Reports a mouse button held; used while dragging settings sliders.                 |
| <code>IsMouseButtonPressed</code>   | Reports a new mouse click; used to activate UI controls.                           |
| <code>IsMouseButtonReleased</code>  | Reports a mouse-button release; used to finish slider dragging.                    |
| <code>SetMouseCursor</code>         | Changes the pointer shape when entering/leaving interactive screens.               |
| <code>CheckCollisionPointRec</code> | Tests whether the mouse point lies inside a UI rectangle.                          |

### Raylib - drawing

| Library function                         | Simple use in this project                                                                   |
|------------------------------------------|----------------------------------------------------------------------------------------------|
| <code>BeginDrawing</code>                | Begins a window-frame drawing pass.                                                          |
| <code>EndDrawing</code>                  | Finishes and presents a window-frame drawing pass.                                           |
| <code>BeginTextureMode</code>            | Redirects drawing to the off-screen gameplay scene texture.                                  |
| <code>EndTextureMode</code>              | Stops drawing to the off-screen scene texture.                                               |
| <code>BeginShaderMode</code>             | Enables the blur shader while the pause overlay is composed.                                 |
| <code>EndShaderMode</code>               | Disables the active shader.                                                                  |
| <code>BeginBlendMode</code>              | Selects a temporary blend mode for layered artwork.                                          |
| <code>EndBlendMode</code>                | Restores the normal blend mode after layered artwork.                                        |
| <code>ClearBackground</code>             | Fills the current target with a background color.                                            |
| <code>DrawTexturePro</code>              | Draws a texture from a source rectangle into a destination rectangle, with scaling/rotation. |
| <code>DrawRectangle</code>               | Draws a filled rectangle used for overlays and simple highlights.                            |
| <code>DrawRectangleRec</code>            | Draws a filled Rectangle value.                                                              |
| <code>DrawRectangleLinesEx</code>        | Draws an outlined rectangle with a chosen line width.                                        |
| <code>DrawRectangleRounded</code>        | Draws a rounded filled rectangle for cards and controls.                                     |
| <code>DrawRectangleRoundedLinesEx</code> | Draws an outlined rounded rectangle.                                                         |
| <code>DrawCircleV</code>                 | Draws a filled circle at a Vector2 position.                                                 |

| Library function        | Simple use in this project                                                        |
|-------------------------|-----------------------------------------------------------------------------------|
| <b>DrawCircleLinesV</b> | Draws a circle outline for the hint stand marker.                                 |
| <b>DrawRing</b>         | Draws a ring used for circular UI decoration.                                     |
| <b>DrawLineEx</b>       | Draws a line between two points with a chosen thickness.                          |
| <b>DrawPoly</b>         | Draws the triangular hint arrowhead.                                              |
| <b>DrawText</b>         | Draws a simple text string.                                                       |
| <b>DrawTextEx</b>       | Draws text with a selected Font, size, spacing, and color.                        |
| <b>Fade</b>             | Returns a color with its alpha multiplied by a factor for overlays and animation. |

## Raylib - assets and audio

| Library function                  | Simple use in this project                                                 |
|-----------------------------------|----------------------------------------------------------------------------|
| <b>LoadTexture</b>                | Loads an image as a GPU texture.                                           |
| <b>GenTextureMipmaps</b>          | Builds lower-resolution texture levels for smoother scaling.               |
| <b>SetTextureFilter</b>           | Chooses texture sampling quality such as bilinear or trilinear filtering.  |
| <b>UnloadTexture</b>              | Releases a GPU texture.                                                    |
| <b>LoadFontEx</b>                 | Loads a font at a requested size.                                          |
| <b>UnloadFont</b>                 | Releases a loaded font.                                                    |
| <b>LoadShader</b>                 | Loads the blur shader used by the pause screen.                            |
| <b>GetShaderLocation</b>          | Finds the shader uniform location for texelSize.                           |
| <b>SetShaderValue</b>             | Sends the texel-size vector to the blur shader.                            |
| <b>UnloadShader</b>               | Releases the blur shader.                                                  |
| <b>LoadRenderTexture</b>          | Creates the off-screen texture used to compose gameplay before pause blur. |
| <b>UnloadRenderTexture</b>        | Releases the off-screen scene texture.                                     |
| <b>InitAudioDevice</b>            | Opens Raylib's audio device.                                               |
| <b>CloseAudioDevice</b>           | Closes Raylib's audio device.                                              |
| <b>LoadSound</b>                  | Loads a short sound effect.                                                |
| <b>LoadSoundAlias</b>             | Creates another playable voice that shares the original sound data.        |
| <b>UnloadSound</b>                | Releases the original sound effect.                                        |
| <b>UnloadSoundAlias</b>           | Releases a sound alias voice.                                              |
| <b>PlaySound</b>                  | Starts a sound effect.                                                     |
| <b>StopSound</b>                  | Stops a sound effect voice.                                                |
| <b>SetSoundVolume</b>             | Sets one sound voice's volume.                                             |
| <b>SetSoundPitch</b>              | Sets one sound voice's playback pitch.                                     |
| <b>LoadMusicStream</b>            | Loads a streaming music track.                                             |
| <b>PlayMusicStream</b>            | Starts or resumes a music stream.                                          |
| <b>StopMusicStream</b>            | Stops a music stream.                                                      |
| <b>UpdateMusicStream</b>          | Feeds the active music stream during each frame.                           |
| <b>SetMusicVolume</b>             | Sets the active music stream volume.                                       |
| <b>AttachAudioStreamProcessor</b> | Attaches the custom muffle callback to gameplay music.                     |
| <b>DetachAudioStreamProcessor</b> | Removes the custom muffle callback at shutdown.                            |

| Library function       | Simple use in this project                                     |
|------------------------|----------------------------------------------------------------|
| <b>SetMasterVolume</b> | Sets the global audio volume used for the master sound toggle. |

## Raylib - text helpers

| Library function     | Simple use in this project                                     |
|----------------------|----------------------------------------------------------------|
| <b>MeasureText</b>   | Measures simple text width for layout calculations.            |
| <b>MeasureTextEx</b> | Measures text with a selected font, size, and spacing.         |
| <b>TextFormat</b>    | Formats a value into a temporary Raylib text string.           |
| <b>TextCopy</b>      | Copies formatted text into a writable buffer.                  |
| <b>TextLength</b>    | Returns the length of a Raylib text string.                    |
| <b>TextIsEqual</b>   | Compares two text strings.                                     |
| <b>TextSplit</b>     | Splits compact route text into tokens for cheatsheet parsing.  |
| <b>TraceLog</b>      | Writes a warning when an asset or audio file cannot be loaded. |

## C standard library, Raymath, math, and compiler built-ins

| Library function            | Simple use in this project                                            |
|-----------------------------|-----------------------------------------------------------------------|
| <b>fopen</b>                | Opens the progress/settings files for reading or writing.             |
| <b>fclose</b>               | Closes a progress/settings file.                                      |
| <b>fprintf</b>              | Writes versioned save data to a file.                                 |
| <b>fscanf</b>               | Reads versioned save data from a file.                                |
| <b>malloc</b>               | Allocates the hint solver hash table.                                 |
| <b>realloc</b>              | Grows the hint solver BFS queue.                                      |
| <b>free</b>                 | Releases dynamic hint solver memory.                                  |
| <b>expf</b>                 | Calculates the exponential smoothing factor in the audio filter.      |
| <b>sinf</b>                 | Creates the pulsing animation used by hint and notice graphics.       |
| <b>atan2f</b>               | Converts the hint push direction into an arrow rotation angle.        |
| <b>Clamp</b>                | Clamps loaded settings values to the 0.0-to-1.0 range.                |
| <b>__builtin_ctzll</b>      | Finds the first set bit when canonicalizing a reachable-cell mask.    |
| <b>__builtin_popcountll</b> | Counts set bits to verify that a level has matching goals and crates. |

## 5. Quick reading guide

When tracing a feature, start at the screen branch in main, then follow the project function named in the table. For movement, inspect BoardUpdate and BoardMovePlayer. For a completed level, follow CheckSolved to ScoreFileRun and SaveProgress. For visual output, follow the screen Draw function into shared UI helpers and finally the Raylib drawing calls.

The Source column uses the form path:line. Because the report is generated from the current workspace, those references are meant to be checked directly against the current files if the code changes later.

End of reference.